The record-swap attack

An insider with database write access moves Bob's ciphertext into Alice's row. The application, reading Alice's row, decrypts for Alice.

1 · STORAGE, ADVERSARY CAN WRITE

ROW	FIELD	CELL
Patient/alice	note	ct_A (Alice)
Patient/bob	note	ct_B (Bob)

copy

AFTER THE WRITE

Patient/alice	note	ct_B (Bob)

Nothing about ct_B says whose it is. Only the row key, which the adversary controls, does.

Same move works across tenants and fields.

2 · APPLICATION

Reads Alice's row.

Builds the context from what it knows to be true:

```
tenant org:st-marys
patient Patient/alice
resource Observation
field note[*].text
```

Calls decrypt(ct_B, context).

What happens next depends entirely on the scheme.

3 · OUTCOME BY SCHEME

AES-256-CBC + HMAC, one shared key

✕ **ACCEPTED: Bob's note shown as Alice's**

The MAC proves the bytes were made under the key, not that they belong to this row. Typical legacy design.

AES-256-GCM, key per tenant, no AAD

✕ **ACCEPTED: same tenant key, tag verifies**

Confidentiality is perfect; binding is absent. Only a cross-tenant move fails, because the key differs.

CPDPM (this work)

✓ **REJECTED: DecryptionError**

K_ctx and the AAD are derived from Alice's context. ct_B was made under Bob's. Both checks fail.

MEASURED, paper/evaluation/record_swap.py

SCHEME	PATIENT SWAP	TENANT MOVE	FIELD MOVE	STALE REPLAY AFTER RETIREMENT
AES-256-CBC + HMAC, one shared key	accepted	accepted	accepted	accepted
AES-256-GCM, key per tenant, no AAD	accepted	rejected	accepted	accepted
CPDPM (this work)	rejected	rejected	rejected	rejected